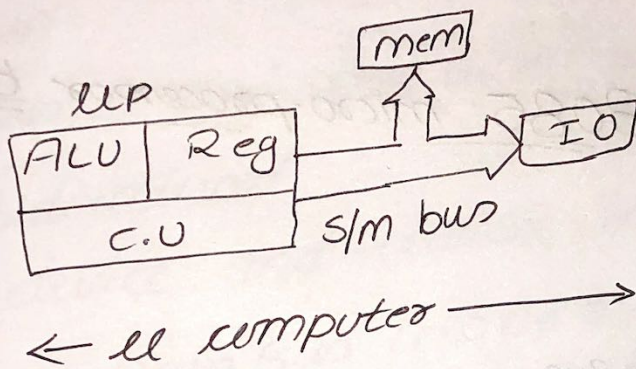


Microprocessor

CPU built into a single chip or VLSI chip is called microprocessor.

Microprocessor is multipurpose, programmable, clock driven register based semiconductor device.

It takes input from input device in binary form and processes it according to given instruction and produces output in binary form.



If we replace CPU by μP in digital computer, it becomes μ computer.

Microprocessor contains ALU, Registers and Control Unit.

ALU (Arithmetical & Logical Unit)

It performs all the arithmetic and logical related operations such as addition, sub, multi, div, AND, OR, NOT, XOR.

Registers

These are group of flip flops which are used to store temporary data during execution of a program.

Control Unit

• It controls all the associated devices connected to microprocessor using control signal.

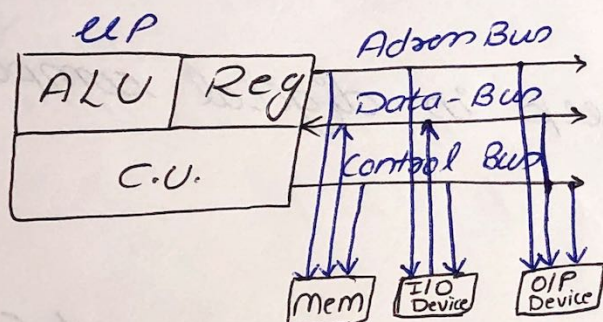
These devices are connected with μp with the help of system bus to form microcomputer

In Assembly Language.

By default hexa-decimal numbers

$$\begin{cases} 1KB = 2^{10}B \\ 1MB = 2^{20}B = 2^{10}KB \\ 1GB = 2^{30}B = 2^{10}MB \end{cases}$$

Bus Organisation of 8085 micro-processor based micro-computer



• Microprocessor is connected with other devices with the help of system bus.

BUS

• It is group of conducting wires which carries information or bit pattern.

• It is of three types:-

Address Bus

• It is group of conducting wires which carries address only.

μp	Address Bus	Data Bus
8085	16 bit	8 bit
8086	20 bit	16 bit

• Memory capacity = $2^{\text{Address Bus}}$

Eg: 8085 $\Rightarrow$ m.cap = $2^{16} = 64 \text{ KB}$

8086 $\Rightarrow$ m.cap = $2^{20} = 1 \text{ MB}$

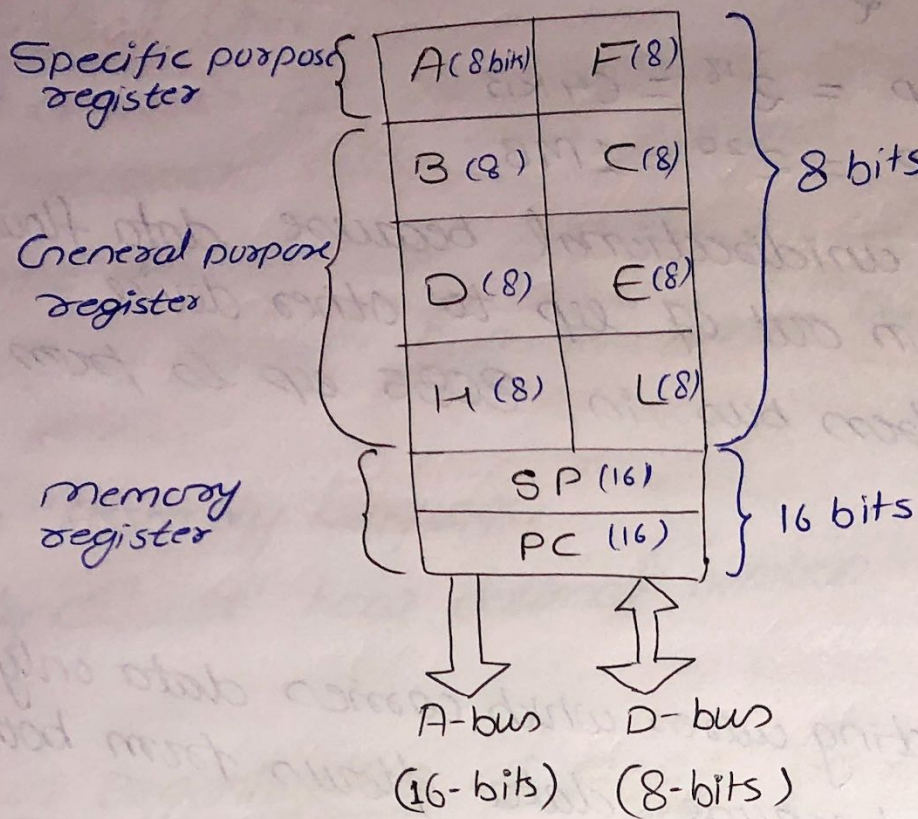
- Address bus is unidirectional because data flows in only one direction out of μp to other device.
- The range of address bus in 8085 μp is from 0000 to FFFF

Data Bus

- It is group of conducting wires which carries data only.
- It is bidirectional because data flows from both μp to device and device to μp .
- Size of Data Bus in 8085 μp is 8-bit. It means it can process 8 bits at a time and most of the registers are designed to operate in 8 bit operation.
- Range of data bus in 8085 from 00 to FF.

Programming Model of 8085 μp .

~~Instruction~~ ^{Information} needed to write an Assembly language program is called programming model.



Types of Register in 8085 :-

1) General purpose register

- These are used to store temporary data during execution of a program.
- There are 6 general purpose registers in 8085 and these are B, C, D, E, H and L and size of each register is 8 bits.
- To perform 16 bit operation, we combine two registers to form register pair (Rp).

$$R_p \rightarrow BC/DE/HL/AF^*$$

↓
PSW

- In this B, D & H are higher order registers and C, E & L are lower order registers.

Indirect Memory M

It is a memory location whose address is specified by contents of HL register pair.

$M \leftarrow [HL]$

H = 20
L = 50

$M \leftarrow 2050$

2) Memory Register

These are used to hold address and size of memory register is 16 bit because length of address bus of 8085 is 16 bit.

And these are SP (Stack Pointer) and PC (Program Counter).

Stack Pointer

It points a memory location in read/write memory.

And it is used in stack related operations such as push and pop.

Program Counter

It holds address of next instruction which we have to fetch.

And PC is incremented by n if size of that instruction is n byte. where $n = 1, 2, 3$ (8085)

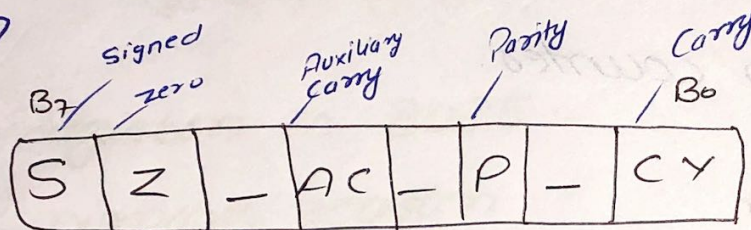
Specific Purpose Registers

Accumulator (A)

For all the arithmetical & logical operation destination operand must be accumulator.

Flag Register (F)

There are five flags in 8085 and these become set (1) or reset (0) depending upon the condition of a result after any ALU operation



3 are undefined

Format of 8085 F.R

Sign Flag:- This flag becomes set when msb D₇ is 1 (negative number) otherwise reset.

SF → 1 → -ve → 80 → FF

SF → 0 → +ve → 00 → 7F

Zero Flag:- This flag becomes set (1) when result is 00H otherwise reset.

ZF → 1 → 00

ZF → 0 → 01 to FF

Auxiliary Carry Flag:- After any ALU operation if D₃ generates any carry and it passes to D₄ this flag becomes set otherwise reset.

It is used in BCD number system.

Parity Flag:- There are two types of parity Even and Odd.

In result if there is even 1's Even parity.

In result if there is odd 1's Odd parity.

It becomes set when there is even parity.

It becomes reset when there is odd parity.

Carry Flag:- This flag becomes set when carry is generated. If D₇ generates any carry and it passes to D₈ otherwise reset.

Q. After execution of following program what will be value of accumulator and status of Flag register.

MVI B 75
MVI A 6D
ADD B
HLT

AC → 1
CY → 0
SF → 1
PF → 1
ZF → 0

75
<u>6D</u>
18
<u>-16</u>
ACC → E 2

Types of instructions of 8085 based upon their functioning

MOV D, S D = Destination operand
 S = Source operand
 MOV = opcode

Opcode:- Operation to be performed
eg. MOV

Operands:- Operation to be performed on

- Size of both operands must be same
- Both operands can be register
- One operand can be register and another can be memory, but both cannot be memory.

(i) MOV R₀, R_s R₀/R_s → A/B/C/D/E/H/L
 R₀ = R_s & R₀ ≠ R_s

(ii) MOV M, R_s
 MOV M, B M ← B
 [HL]

(iii) MOV R₀, M R₀ ← M_c (M_c = contents of M)

(iv) MVI R₀/M 8 bit data

MVI B 75 B = 75

↑
Immediate data ← constant

In case of register pair we only mention higher order register such as B/D/H in any instruction as any operation. operand.

LDA $addr^s$ $A \leftarrow addr^s$

LDA 2050 → $A \leftarrow \boxed{2050}$

STA $addr^s$

STA 3050

$A \rightarrow \boxed{3050}$

STA $addr^s$ transfers contents of A register into a memory location. which is directly specified in the instruction as an operand.

LXI H 2050 → HL = 2050

LXI R_p ~~2050~~
16 bit data

LXI Register Pair 16 bit data

LXI B 3050 B = 30
 C = 50

LHLD $addr^s$
LHLD 2050

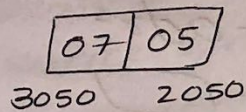
$L \leftarrow addr^s$
 $H \leftarrow addr^s + 1$

$L \leftarrow 2050$
 $H \leftarrow 2051$

SHLD $addr^s$

$addr^s \leftarrow L$
 $addr^s + 1 \leftarrow H$

Write an Assembly Language Program in 8085 which swap contents of two memory locations 2050 & 3050.



$B \leftarrow 2050$

$2050 \leftarrow 3050$

$3050 \leftarrow B$

LDA 2050

$A \leftarrow 05$

MOV B, A

$B \leftarrow 05$

LDA 3050

$A \leftarrow 07$

STA 2050

$2050 \leftarrow 07$

MOV A, B

$A \leftarrow 05$

STA 3050

$3050 \leftarrow 05$

HLT

LDAX Rp

$R_p \rightarrow BC/DE/HL$

LDAX B

$A \leftarrow \boxed{BC}$

It copies contents of a memory location into Accumulator and that memory location is indirectly specified by contents of register pair.

STAX Rp

LXI D 3050

$DE = 3050$

MVI A 75

$A = 75$

STAX D

$\boxed{75}$
3050

HLT

XCHG

It exchanges contents of HL register pair with DE register pair.

LXI H 3050

LXI D 7020

$H = 70, L = 20$

XCHG

$D = 30, E = 50$

→ Input Output port address of 8085 is 8 bit.
Ranging from 00 to FF.

IN 8 bit port address

MVI A F2

OUT 45

IN 45

A ← 45

45

F2

Arithmetic Instruction (one operand must be Accumulator)

ADD R

$A = A + R$

ADD M

$A = A + M$

ADI Data

$A = A + \text{Data}$

ADI 75 $A = A + 75$

ADC R

$A = A + R + (CY)_{pr}$

ADC M

$A = A + M + (CY)_{pr}$

ACI Data

$A = A + \text{Data} + (CY)_{pr}$

Q. Write an Assembly level program which determine addition of ^{two} 8 bit number.
Also determine status of flag register

MOV A, L

$A \leftarrow 3A, CY = 1$

ADD H

"

STA 3050

MVI A 00

$A = A + 00 + (CY)_{pr}$

$00 + 00 + 1$

ACI 00

STA 3051

HLT

F5 45

2031 2050

01 3A

3051 3050

$\left\{ \begin{array}{l} AC \leftarrow 0 \\ CY \leftarrow 1 \\ SF \leftarrow 0 \\ ZF \leftarrow 0 \\ PF \leftarrow 1 \end{array} \right.$

SUB R $A = A - R$

SUB M $A = A - M$ $\rightarrow$ indirect memory

SUBI Data
8 bit data
 $A = A - \text{Data}$

SBB R $A = A - R - (CY)_{pr}$ $\rightarrow$ previous carry

SBB M $A = A - M - (CY)_{pr}$

SBI Data $A = A - \text{Data} - (CY)_{pr}$

$\rightarrow$ increment
INR R $R = R + 1$

INR M $M = M_c + 1$

INX R_p $R_p = R_p + 1$ (To point next memory location we use INX R_p)

$\rightarrow$ Decrement

DCR R $R = R - 1$

DCR M $M = M_c - 1$

DCX R_p $R_p = R_p - 1$

DAD R_p $HL = HL + R_p$

DAD B $HL = HL + BC$

Q. Write an Assembly level program which determine addition of two 16 bit numbers which are stored from starting memory location 2050 and store the result at starting memory location 3050.

a) Using all the instructions which are related to 8 bit operation only

b) Use all the instruction which are related to 16 bit operation only.

1st no.

29	75
----	----

2051 2050

2nd no.

34	98
----	----

2053 2052

5E	0D
----	----

3051 3050

a) LDA 2050
MOV B, A
LDA 2052
ADD B
STA 3050
LDA 2051
MOV B, A
LDA 2053
ADC B
STA 3051
HLT

b) LHLD 2050

L ← 75
H ← 29

XCHG ED

LHLD 2052

L ← 98
H ← 34

DAD D

SHLD 3050

HLT

Decimal Adjust Accumulator

It stores result in decimal or BCD number system.

If result is greater than 9 add 6.

Q.1 UAP which determine addition of 2 8 bit BCD number

Q.2 Addition of two 16 bit BCD no.

1.

86	75
----	----

2051 2050

01	61
----	----

3051 3050

```
LXI H 2050    M ← 2050
MOV A, M
INX H    M ← 2051
ADD M
DAA
STA 3050
MVI A 00
ACI 00
STA 3051
HLT
```

DAA instruction is applicable only in accumulator and after ADD, ADC, ADI, ACI

2.

29	75
2051	2056

34	98
2053	2052

5E	0D
3051	3050

```

LDA 2050
MOV B, A
LDA 2052
ADD B
DAA
STA 3050
LDA 2051
MOV B, A
LDA 2053
ADC B
DAA
STA 3051
HLT

```

Logical Instruction
Destination operand must be accumulator.
In this we perform operation bit by bit.

ANA RIM $A = A \text{ AND } RIM$
ANI Data $A = A \text{ AND Data}$

$A \rightarrow \overset{xy}{45}$
ANI 0F
ANI F0

$A \leftarrow 05$ } marking
 $A \leftarrow 40$ } of digits

0100	0101	
0000	1111	
		(05)
0000	0101	

After execution of ANA / ANI instruction
AC=1, CY=0 & S, Z, P modify based upon result.

ORA RIM $A = A \text{ OR RIM}$
ORI Data $A = A \text{ OR Data}$

Q. write down any four instructions which are used to reset Accumulator

1- MVI A 00
2- ANI 00
3- XRA A ($A = A \oplus A$)
4- SUB A ($A = A - A$)

CMA $A = \bar{A}$

It is used to determine one's complement of a number.

It does not affect status of any flag register.

Q. WAP which determine 2's complement of 8 bit number which is stored in 2050 & store the result in 3050.

LDA 2050

CMA

INR A

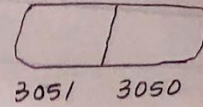
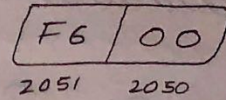
STA 3050

HLT

8. WAP which determine 2's complement of a 16 bit number.

```
LDA 2050
CMA
ADI 01
STA 3050
LDA 2051
CMA
ACI 00
STA 3051
HLT
```

```
LHLD 2050
MOV A, L
CMA
MOV L, A
MOV A, H
CMA
MOV H, A
INX H
SHLD 3050
HLT
```

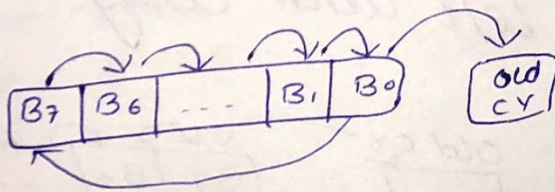


INR A does not affect status of CY flag.
while ADI 01 affect CY flag.

Rotate Instruction

RRC

Rotate Accumulator Right without carry.



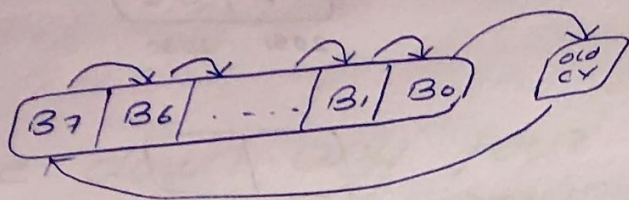
$$\text{New } B_n = \text{Old } B_{n+1}$$

$$\text{New } B_1 = \text{New CY} = \text{old } B_0$$

After applying RRC instruction 4 times, number becomes reverse. Number should be available in Accumulator.

RAR

Rotate accumulator right with carry.

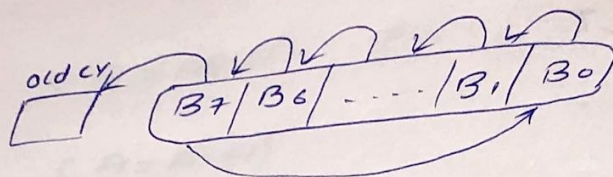


$$\text{New } B_n = \text{Old } B_{n+1}$$

$$\text{New } B_7 = \text{Old } B_{cy}$$

$$\text{New } cy = \text{Old } B_0$$

RLC



$$\text{New } B_n = \text{Old } B_{n+1}$$

$$\text{New } cy = \text{Old } B_7$$

$$\text{New } B_0 = \text{Old } B_7$$

After applying RLC four times contents of Accumulator becomes reverse

$$A = A \times 2^n$$

n = no. of times RLC applied

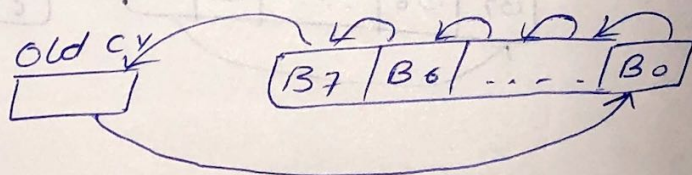
RAL

Rotate Accumulator left with carry.

$$\text{New } B_n = \text{Old } B_{n-1}$$

$$\text{New } cy = \text{Old } B_7$$

$$\text{New } B_0 = \text{Old } cy$$



8. WAP which determine sum of digits of 8 bit number which is stored in memory location 2050 & store the result at memory location 3050.

2050
75

3050
10C

```

LDA 2050    A ← 75
MOV B, A    B ← 75
ANI 0F      A ← 05
MOV C, A    C ← 05
MOV A, B    A ← 75
ANI FO      A ← 70
RLC
RLC
RLC
RLC    A ← 07
ADD C
STA 3050
    
```

Branching Instruction

• Unconditional Jump Instruction

JMP addr PC = addr

```

2000 LDA 2050
2003 MVI B 75
2005 ADD B
2006 JMP 2000
PC
    
```

Program control is transferred to a memory location which is specified as an operand.

It is immediate addressing mode.
3 bit instruction.

2. Conditional Branching Instruction

In this program control is transferred to a memory location which is specified in the instruction only if condition is satisfied.

J codⁿ addr Jump at addr if condition is satisfied.

JC addr^s Jump at addr if $CY=1$

JNC addr^s Jump at addr if $CY=0$

JZ addr^s Jump at addr if $ZF=1$

JNZ addr^s Jump at addr if $ZF=0$

JPE Jump at addr if $PF=1$

JPO Jump at addr if $PF=0$

JM Jump at addr if $SF=1$ (-ve)

JP Jump at addr if $SF=0$ (+ve)

8. Write an assembly level program which determine addition of 2 8bit numbers. if result is greater than 7F store ~~a~~ result at 3050 otherwise at memory location

3090.

00 to 7F (+ve) $\rightarrow SF \rightarrow 0$
80 to FF (-ve) $\rightarrow SF \rightarrow 1$

LHLD 2050

MOV A, L

ADD H

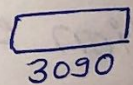
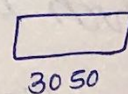
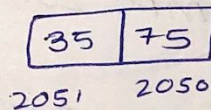
JM * Jump at * addr if $SF=1$ ($>7F$)

STA 3090

JMP #

* STA 3050

HLT



Q. WAP which determine subtraction of two 8-bit numbers if any carry is generated store the result at 3050 ~~else~~ at PORT F5.

```
LHLD 2050
MOV A, H
SUB H
JC *
OUT F5
JMP #
* STA 3050
# HLT
```

Addressing modes of 8085.

The way of specifying data to be operated by an instruction is called addressing mode.

Types of addressing modes

1. Register addressing mode.

The data to be operated are available inside register.

eg:

MOV A, L	ANA D
ADD B	XRA C
SUB H	DAD

2. Immediate addressing mode

Source operand will be always data

eg:

MVI B 75	SBI 75	JMP
AOI 75	XRI	JC
SUI 75	ANI	

3. Direct Addressing mode

In this data to be operated is available inside a memory location and that memory location is directly specified in the instruction as an operation.

eg: LDA 2050
LHLD 2050
STA 3050
SHLD 3050

4. Register Indirect addressing mode.

In this data to be operated is available inside a memory location & that memory location is indirectly specified by contents of register pair.

MOV M, R_s STAX R_p
MOV R_o, M ADD M
LDAX R_p

5. Implied/Implicit addressing mode

In this operand is hidden & data to be operated is available inside instruction itself.

DAA RRC
CMA RAR
RLC RAL

{ HLT ⇒ machine control, ^{statement} ~~instruction~~ }
Not a addressing mode

Q. WAP which multiplies 2 numbers 5 & 3 which are available in register B & C resp. & store the result in memory location 3050.

MVI B 05

MVI C 03

MVI A 00

* ADD B

DCR C

JNZ *

STA 3050

HLT

STC Set carry flag $CY = 1$

CMC $CY = \overline{CY}$

Q. WAP which converts an 8 bit number into gray number.

D7
2050

LDA 2050 $A \leftarrow D7$

MOV B, A $B \leftarrow D7$

RAR $A \leftarrow GB$

XRA B

STA 3050

HLT

CMP R	A-R	SF	A > R	A = R	A < R
CMP M	A-M	ZF	0	0	1
CPI Data	A-Data	CY	0	1	0
			0	0	1

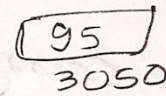
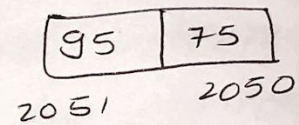
It only performs subtraction operation and does not store result like SUB instr and it modifies sign, zero & carry flag.

Q. WAP which determine larger of two numbers.

```

LHLD 2050
MOV A, H
CMP L      A-L
JNC *
MOV A, L
* STA 3050
HLT

```



Q. WAP which determine smaller of two numbers

```

LHLD 2050
MOV A, H
CMP L
JC *
MOV A, L
* STA 3050
HLT

```

8. WAP which determine sum of series of even no. in an array of five numbers.

98	75	F2	42	35
2054	2053	2052	2051	2050
				↑ M

```

MVI C 05
LXI H 2050
MVI B 00
MVI A, M
# ANI 01
JNZ *
MOV A, B
ADD M
MOV B, A
* INX H
    DCR C
    JNZ #
    MOV L, B
    MOV H, D
    SHLD 3050
    
```

JNC *
INR D

Stack and subroutine

Stack pointer is initialized using LXI SP addr instruction

It is a memory location available in read write memory & it is based upon LIFO property with the help of push instruction we transfer contents of register pair into stack and this data is retrieved back into register pair with the help of pop instruction.

LXI SP addr

PUSH R_P

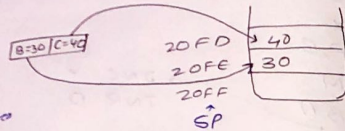
$R_p \rightarrow BC/DE/HL/PSW (A \& F)$
 $\downarrow$
 Program Status Word

LXI SP 20FF

LxI B 3040

PUSH B

Storing of elements takes place from SP-1 address.



$SP = SP - 1$ higher_order_registers
2 (B)

SP ← H. O. R (B)

$$SP = SP - 1$$

SP ← L.O.R (C)

PUSH PSW

$$SP = SP^{-1}$$
$$SP \leftarrow A$$
$$SP = SP - 1$$
$$SP \leftarrow F$$

POP R_p

$LOR \leftarrow SP$

$$SP = SP + 1$$

HOR $\leftarrow$ SP

$$SP = SP + 1$$

POD D

$$E \leftarrow SP$$
$$SP = SP + 1$$
$$D \leftarrow SP$$
$$SP = SP + 1$$

Q. Implement XCHG instruction using push & pop instruction.

LXI SP 3099

LX I H 2050
2060

LXI D 3060

PUSH H

PUSH D

POP H

POP D

HLT

Q. WAP in 8085 which determine 2's complement of flag register.

LXI SP 3099

PUSH PSW

POP B

MOV A,C

CMA

$$= \text{INR } A$$

mov C, A

PUSH B

POP PSW

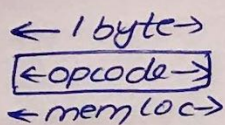
HALT

Types of instruction on the basis of word length or bytes consumed in memory

Every instruction has first byte as opcode

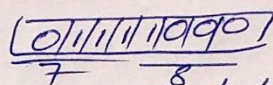
1 byte instruction

This type of instruction has only 1 byte and that byte is opcode.



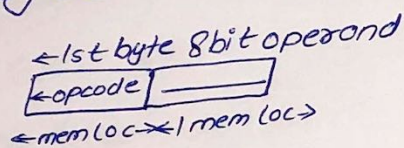
MOV A, B
2000: 78

MOV A, C
MOV A, M



Neither 8 bit ^{data} nor 16 bit address or data is present as an operand.

2 byte instruction

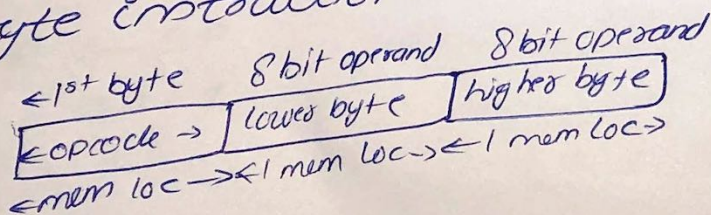


2000: MVI B 75
2000: opcode
2001: 75

2000: OUT 45
2000: opcode
2001: 45

Either 8 bit data or address will be present as an operand.

3 byte instruction



2000: LDA 2050
2000: opcode of LDA
2001: 50
2002: 20

LXI H 3050

SHLD 3050

In this either 16 bit data or address will be present as an operand.

Subroutine

It is group of instructions which are written separate from main program & used to perform any task repeatedly. (reusability)

2000: LXI SP 3099

2003: MVI C 00

2005: OUT 45

2007: CALL 2070

200A: INR C

JMP #

2070: MVI B FF

* DCR B

JNZ *

RET

It is implemented using call & return instruction.

CALL addr

After execution of this instruction program control is transferred to a sub-routine & this subroutine address is mentioned in the instruction as an operand.

And value of program counter (addr of next instruction after call) is transferred into stack.

{CALL = PUSH + JMP}

Difference between CALL & JMP instrⁿ

CALL

- Program control is transferred from one memory location in main program to another memory location which is not available in main program.

JMP

- Program control is transferred from one memory location in main program to another memory location in main program.

CALL

- It uses stack pointer
- Stack pointer is decremented by 2
 $SP = SP - 2$
- To come back in main program RET is used
- more T-states are required (18 T)

JMP

- It does not use stack pointer
- constant is not used
 $SP = SP$
- No need of RET
- Less T-states are required. (10 T)

RET

It means end of subroutine and after execution of this instr program control is transferred to main program and value of PC is retrieved.

{ RET = POP + JMP }

After execution of this stack pointer is incremented by 2.

Conditional CALL

- CC $addr^s$ call at $addr^s$ only if $CY = 1$
- CNC $addr^s$ call at $addr^s$ only if $CY = 0$
- CZ $addr^s$ call at $addr^s$ only if $ZF = 1$
- CNZ $addr^s$ call at $addr^s$ only if $ZF = 0$
- CPE $addr^s$ call at $addr^s$ only if $PF = 1$
- CPO $addr^s$ call at $addr^s$ only if $PF = 0$
- CM $addr^s$ call at $addr^s$ only if $SF = 1$
- CP $addr^s$ call at $addr^s$ only if $SF = 0$

Conditional RET

RC Return from sub-routine only if $CY=1$
RNC Return from SR only if $CY=0$
RZ Return from SR only if $ZF=1$
RNZ Return from SR only if $ZF=0$
RPE Return from SR only if $PF=1$
RPO Return from SR only if $PF=0$
RM Return from SR only if $SF=1$
REP Return from SR only if $SF=0$

SPHL $SP = HL$

It transfers contents of HL Register pair into SP.

$LXI\ SP\ 3099$
OR
 $LXI\ H\ 3099$
SPHL

PCHL $PC = HL$

Transfer contents of HL Register pair into PC.

$PC = 2080$
JMP 2080
OR
 $LXI\ H\ 2080$
PCHL

XTHL

It exchanges two bytes available in top of stack with HL Register pair.

LHLD 2050
SPHL
LHLD 2052
DAD SP
SHLD 3050

Addition of 2 16bit number using minimum instructions

Determine largest number in an array of n numbers. Assume that numbers are stored from 2051 address and value of count is stored in 2050 and store the result at 3050.

2050 2051 2052 2053 2054 2055

5	13	41	28	F7	39
---	----	----	----	----	----

LXI H 2050

MOV C, M

DCR C

INX H

MOV A, M

#INX H
CMP M

JNC *

MOV A, M

*DCR C

JNZ #

STA 3050

Memory mapped I/O

- i) $IO/\bar{M} = 0$
- ii) Address Bus = 16 bits
0000 to FFFF
- iii) $\overline{MEMRD}$, $\overline{MEMWR}$
- iv) Arithmetic & logical operation can be performed.
- v) Any register can be involved
- vi) Addressing mode:
Direct, Indirect, Immediate
- vii) DMA transfer is possible.

I/O mapped I/O

- i) $IO/\bar{M} = 1$
- ii) Address Bus = 8 bit
00 to FF
- iii) $\overline{IORD}$, $\overline{IOWR}$
- iv) Arithmetic & logical operation cannot be performed.
- v) Only accumulator is used.
- vi) Addressing mode:
Direct
- vii) DMA transfer is not possible.

Q. WAP which determine square root of a number.

```

LDA 2050
MVI B, 01
MOV C, A
*JZ #
MOV A, C
SUB B
INR B
INR B
MOV C, A
JMP *
#MOV A, C
STA 3050
HLT

```

```

MVI C, 01
MVI B, 01
LDA 2050
#SUB B
JZ *
INR B
INR B
INRC
JMP #
*MOV A, C
STA 3050
HLT

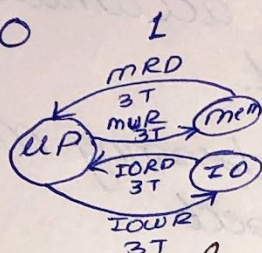
```

```

      1617
  ①  -1
     15
  ②  -3
     12
  ③  -5
     7
  ④  -7
     0

```

$\overline{IO/\overline{M}}$	S_1	S_0	Operation (m/c cycle)	T-states
0	1	1	OP-fetch	4T
0	1	0	MRD	3T
0	0	1	MWR	3T
1	1	0	IORD	3T
1	0	1	IOWR	3T



Q. Determine total number of T-states & machine cycle to execute MOV A, B instruction.

MOV A, B

First byte is always opcode so first machine cycle will be opcode-fetching

2000: opcode	m/c op fetch	T-states 4T
$A \leftarrow B$	No	

Only 1 m/c cycle is required.

MOV B, M	
2000: opcode	Op-fetch - 4T
$B \leftarrow M$	Mem RD - 3T
	<u>7T</u>

STAX B

2000: opcode op-fetch - 4T

A → $\boxed{\text{BC}}$ - MWR - $\frac{3T}{7T}$

LDA 2050

2000: opcode op-fetch - 4T

2001: 50 - MRD - 3T

2002: 20 - MRD - 3T

A ← 2050 - MRD - $\frac{3T}{13T}$

IN 45 2000: opcode-fetch - 4T

2001: 45 MRD - 3T

A ← 45 $\boxed{}$ IORD - $\frac{3T}{10T}$

2000: opcode fetch → 4T

2001: 30 → MRD 3T

2002: 30 MRD 3T

L → 3050 → MWR → 3T

H → 3051 → MWR → 3T

16T

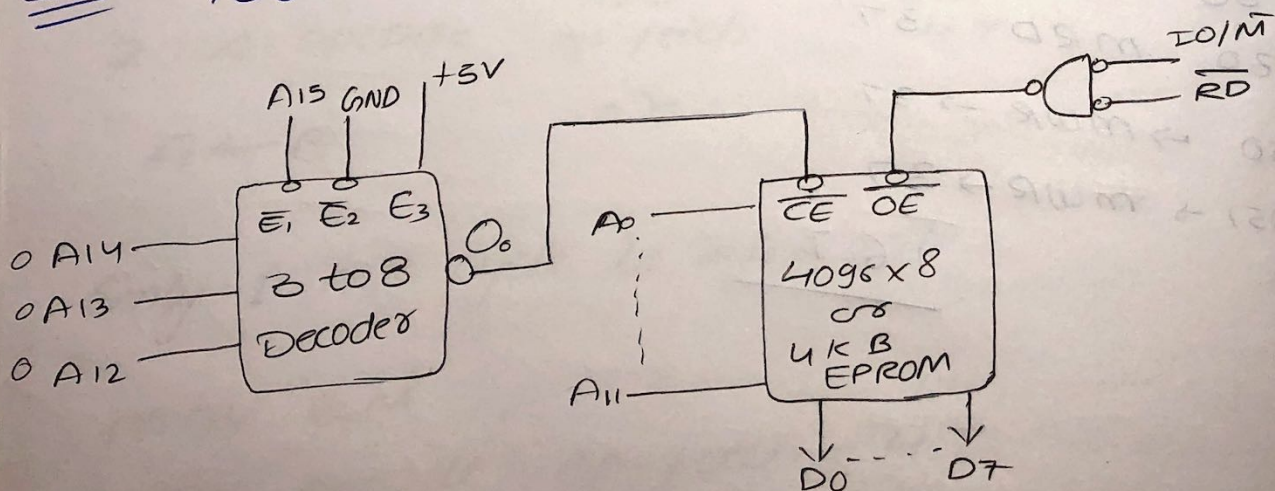
- With the help of formula $\text{Capacity} = 2^{A.L.}$ determine no. of address line which are required to connect with memory chip.
- Remaining lines of address bus ($A_0 - A_{15}$) are connected to decoder or NAND gate.
- Generate control signal $\overline{\text{memory Read}} (\overline{\text{mRD}})$
 $\overline{\text{memory Write}} (\overline{\text{mWR}})$
- To determine addressing range minimum value of bits of lines that are connected to memory chip will be 0 and maximum value 1.

Q. Interface 4KB (4096×8) EPROM with 8085

$\downarrow$
 $N \times 5$
 (No. of registers) (Size of each register)

also determine addressing range.

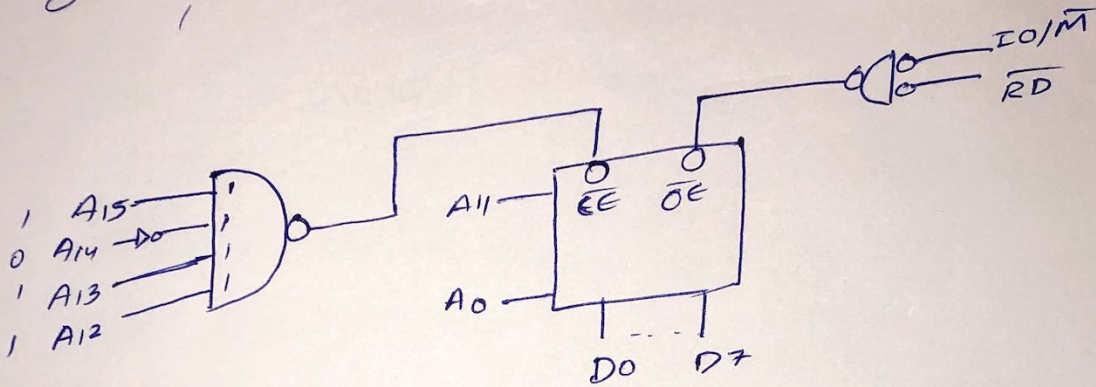
Solⁿ $4096 = 2^{12} = 2^{A.L.}$



Range 0000 to 0FFF

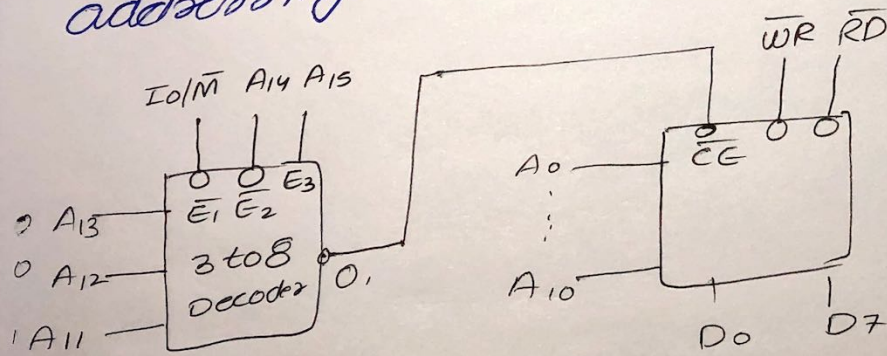
In the above circuit if O_0 is replaced by O_5 then what will be addressing range.
 Range 5000 to 5FFF

A_{15}	A_{14}	A_{13}	A_{12}	$A_{11} \ A_{10} \ \dots \ A_0$		
0	ϕ	0	ϕ	0000	0000	0000
0	ϕ	0	ϕ	1111	1111	1111



RAM Interfacing

- Interface 2KB RAM with 8085 also determine addressing range.



A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	$A_{10} \ \dots \ A_0$		
1	0	0	0	1	0000	0000	000
1	0	0	0	1	1111	1111	111

8800 to 8FFF